

---

# TP 2 : Hiérarchies, Transformations et Matériaux

AR/VR Lab

---

## Objectifs du TP

1. Comprendre et utiliser le système de coordonnées Local vs World.
2. Manipuler les relations Parent-Enfant dans Unity.
3. Utiliser `Instantiate` pour créer des objets dynamiquement.
4. Créer un système solaire interactif simple.

## Partie 1 : Le Système Solaire

1. Créez une nouvelle Scène (`File > New Scene > Basic (Built-in)`). Sauvegardez-la sous `SolarSystem`.
2. Créez une Sphère au centre (0, 0, 0) nommée "Soleil". Scale (2, 2, 2).
3. Créez un Matériau "Mat\_Soleil" (Jaune, Emission activée pour qu'il brille). Appliquez-le.
4. Créez une Sphère nommée "Terre". Scale (1, 1, 1). Position (5, 0, 0).
5. Créez un Matériau "Mat\_Terre" (Bleu). Appliquez-le.

## Hiérarchie Interactive

1. Créez une petite Sphère "Lune". Scale (0.3, 0.3, 0.3). Position (6, 0, 0).
2. Dans la **Hierarchy**, glissez "Lune" sur "Terre". "Lune" devient enfant de "Terre".
3. Glissez "Terre" sur "Soleil". "Terre" devient enfant de "Soleil".
4. **Test** : Sélectionnez le Soleil et faites-le tourner (Rotation Y). Tout le système tourne !
5. **Test** : Sélectionnez la Terre et faites-la tourner. La Lune tourne autour de la Terre.
6. *Remettez les rotations à 0 avant la suite.*

**Scripting d'Orbite** Nous allons automatiser cela.

1. Créez un script `Orbit.cs`.
2. Ce script doit faire tourner l'objet autour d'un pivot.

```
1 using UnityEngine;
2
3 public class Orbit : MonoBehaviour
4 {
5     public Transform pivot; // L'objet autour duquel on tourne (ex: Soleil)
6     public Vector3 axis = Vector3.up; // Axe de rotation (Y par défaut)
7     public float speed = 20f; // Vitesse en degr s par seconde
8
9     void Update()
10    {
11        if (pivot != null)
12        {
13            // Fait tourner "gameObject" autour de "pivot.position"
14            // selon l'axe "axis" a la vitesse "speed"
15            transform.RotateAround(pivot.position, axis, speed * Time.
16            deltaTime);
17        }
18        else
19        {
20            // Si pas de pivot, on tourne sur soi-meme (Rotation locale)
21            transform.Rotate(axis, speed * Time.deltaTime);
22        }
23    }
```

1. Attachez `Orbit` à la **Terre**.
  - Pivot : Glissez le Soleil.
  - Speed : 10.
2. Attachez `Orbit` au **Soleil**.
  - Pivot : Aucun (None).
  - Speed : 5. (Il tournera sur lui-même).
3. Attachez `Orbit` à la **Lune**.
  - Pivot : Glissez la Terre.
  - Speed : 50.

**Lancez le jeu. Vous devriez voir un système solaire fonctionnel !**

## partie-2-instantiation-dynamique-pluie-de-muxe9tuxe9orites

Nous allons faire apparaître des objets aléatoirement.

1. Créez un script `MeteorSpawner.cs`.
2. Créez un **Prefab** de météorite :
  - Créez un Cube ou une Sphère.
  - Ajoutez un Rigidbody.
  - Créez un dossier "Prefabs" dans Project.
  - Glissez l'objet de la Hierarchy vers le dossier Prefabs.
  - Supprimez l'objet de la scène.

```

1 using UnityEngine;
2
3 public class MeteorSpawner : MonoBehaviour
4 {
5     public GameObject meteorPrefab; // Le modele a copier
6     public float spawnInterval = 1.0f; // Temps entre chaque apparition
7     public float spawnHeight = 10f; // Hauteur d'apparition
8
9     private float timer = 0f;
10
11     void Update()
12     {
13         timer += Time.deltaTime;
14
15         if (timer >= spawnInterval)
16         {
17             SpawnMeteor();
18             timer = 0f; // Reset du timer
19         }
20     }
21
22     void SpawnMeteor()
23     {
24         // Position aleatoire
25         float x = Random.Range(-10f, 10f);
26         float z = Random.Range(-10f, 10f);
27         Vector3 randomPos = new Vector3(x, spawnHeight, z);
28
29         // Rotation aleatoire
30         Quaternion randomRot = Random.rotation;
31
32         // Creation de l'objet
33         Instantiate(meteorPrefab, randomPos, randomRot);
34     }
35 }

```

1. Créez un Game Object vide dans la scène “Spawner”.
2. Attachez le script.
3. Assignez votre Prefab de météorite dans la case “Meteor Prefab”.
4. Lancez le jeu. Observez la pluie d’objets !

**Exercise** Si on laisse le jeu tourner, les météorites s’accumulent et le PC va ralentir (Memory Leak). **Défi** : Modifiez le script (ou créez-en un nouveau sur le Prefab) pour détruire les météorites après 5 secondes ou quand elles tombent trop bas ( $Y < -10$ ).

Utilisez `Destroy(gameObject, delay);`